IMPORTING LIBRARIES

In [1]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import plotly.express as px
import plotly.graph_objects as go
```

IMPORTING DATASET

In [2]:
```
traffic = pd.read_csv('Banglore_traffic_Dataset.csv')
```

In [3]:
```
traffic.head()
```

Out[3]:

	Date	Area Name	Road/Intersection Name	Traffic Volume	Average Speed	Travel Time Index	Congestion Level	Road Capacity Utilization	Incident Reports	Environmental Impact	Public Transport Usage	Traffic Signal Compliance	Parking Usage	Pedestrian and Cyclist Count	Weather Conditions	Roadwork and Construction Activity
0	2022-01-01	Indiranagar	100 Feet Road	50590	50.230299	1.500000	100.000000	100.000000	0	151.180	70.632330	84.044600	85.403629	111	Clear	No
1	2022-01-01	Indiranagar	CMH Road	30825	29.377125	1.500000	100.000000	100.000000	1	111.650	41.924899	91.407038	59.983689	100	Clear	No
2	2022-01-01	Whitefield	Marathahalli Bridge	7399	54.474398	1.039069	28.347994	36.396525	0	64.798	44.662384	61.375541	95.466020	189	Clear	No
3	2022-01-01	Koramangala	Sony World Junction	60874	43.817610	1.500000	100.000000	100.000000	1	171.748	32.773123	75.547092	63.567452	111	Clear	No
4	2022-01-01	Koramangala	Sarjapur Road	57292	41.116763	1.500000	100.000000	100.000000	3	164.584	35.092601	64.634762	93.155171	104	Clear	No

EXPLORATORY DATA ANALYSIS

Why do EDA

- Model building
- Analysis and reporting
- Validate assumptions
- Handling missing values
- feature engineering
- detecting outliers

Column Types in our dataset

- Numerical - date, traffic volume, average speed,incident reports, public transport usage, etc
- Categorical - Area name, weatether condtion , construction_activity..
- Mixed- road/intersection name

In [4]:
```
traffic.sample()
```

Out[4]:

	Date	Area Name	Road/Intersection Name	Traffic Volume	Average Speed	Travel Time Index	Congestion Level	Road Capacity Utilization	Incident Reports	Environmental Impact	Public Transport Usage	Traffic Signal Compliance	Parking Usage	Pedestrian and Cyclist Count	Weather Conditions	Roadwork and Construction Activity
436	2022-02-16	Yeshwanthpur	Yeshwanthpur Circle	5034	39.128847	1.175319	12.033866	21.817177	1	60.068	24.010795	72.783577	79.590395	174	Clear	No

In [5]:
```
traffic.describe()
```

Out[5]:

	Traffic Volume	Average Speed	Travel Time Index	Congestion Level	Road Capacity Utilization	Incident Reports	Environmental Impact	Public Transport Usage	Traffic Signal Compliance	Parking Usage	Pedestrian and Cyclist Count
count	8936.000000	8936.000000	8936.000000	8936.000000	8936.000000	8936.000000	8936.000000	8936.000000	8936.000000	8936.000000	8936.000000
mean	29236.048120	39.447427	1.375554	80.818041	92.029215	1.570389	108.472096	45.086651	79.950243	75.155597	114.533348
std	13001.808801	10.707244	0.165319	23.533182	16.583341	1.420047	26.003618	20.208460	11.585006	14.409394	36.812573
min	4233.000000	20.000000	1.000039	5.160279	18.739771	0.000000	58.466000	10.006853	60.003933	50.020411	66.000000
25%	19413.000000	31.775825	1.242459	64.292905	97.354990	0.000000	88.826000	27.341191	69.828270	62.545895	94.000000
50%	27600.000000	39.199368	1.500000	92.389018	100.000000	1.000000	105.200000	45.170684	79.992773	75.317610	102.000000
75%	38058.500000	46.644517	1.500000	100.000000	100.000000	2.000000	126.117000	62.426485	89.957358	87.518589	111.000000
max	72039.000000	89.790843	1.500000	100.000000	100.000000	10.000000	194.078000	79.979744	99.993652	99.995049	243.000000

In [6]:
```
traffic.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8936 entries, 0 to 8935
Data columns (total 16 columns):
 #   Column                              Non-Null Count  Dtype  
---  ------                              --------------  -----  
 0   Date                                8936 non-null   object 
 1   Area Name                           8936 non-null   object 
 2   Road/Intersection Name              8936 non-null   object 
 3   Traffic Volume                      8936 non-null   int64  
 4   Average Speed                       8936 non-null   float64
 5   Travel Time Index                   8936 non-null   float64
 6   Congestion Level                    8936 non-null   float64
 7   Road Capacity Utilization           8936 non-null   float64
 8   Incident Reports                    8936 non-null   int64  
 9   Environmental Impact                8936 non-null   float64
 10  Public Transport Usage              8936 non-null   float64
 11  Traffic Signal Compliance           8936 non-null   float64
 12  Parking Usage                       8936 non-null   float64
 13  Pedestrian and Cyclist Count        8936 non-null   int64  
 14  Weather Conditions                  8936 non-null   object 
 15  Roadwork and Construction Activity  8936 non-null   object 
dtypes: float64(8), int64(3), object(5)
memory usage: 1.1+ MB
```

In [7]:
```
## checking for null values
traffic.isnull().sum()
```

Out[7]:
```
Date                                  0
Area Name                             0
Road/Intersection Name                0
Traffic Volume                        0
Average Speed                         0
Travel Time Index                     0
Congestion Level                      0
Road Capacity Utilization             0
Incident Reports                      0
Environmental Impact                  0
Public Transport Usage                0
Traffic Signal Compliance             0
Parking Usage                         0
Pedestrian and Cyclist Count          0
Weather Conditions                    0
Roadwork and Construction Activity    0
dtype: int64
```

NOTE : THERE IS NO NULL VALUES IN OUR DATASET

In [25]:
```
#ANALYSING TRAFFIC_VOLUME COLUMN
```

In [ ]:
```
traffic[['Area Name','Traffic Volume']].max()
```

In [ ]:
```
traffic[['Area Name','Traffic Volume']].min()
```

In [ ]:
```
traffic[['Area Name','Traffic Volume','Average Speed','Incident Reports']]
```

In [10]:
```
traffic[['Area Name','Traffic Volume','Average Speed','Incident Reports']].describe()
```

Out[10]:

	Traffic Volume	Average Speed	Incident Reports
count	8936.000000	8936.000000	8936.000000
mean	29236.048120	39.447427	1.570389
std	13001.808801	10.707244	1.420047
min	4233.000000	20.000000	0.000000
25%	19413.000000	31.775825	0.000000
50%	27600.000000	39.199368	1.000000
75%	38058.500000	46.644517	2.000000
max	72039.000000	89.790843	10.000000

In [11]:
```
traffic[['Area Name','Traffic Volume','Average Speed','Incident Reports']].max()
```

Out[11]:
```
Area Name           Yeshwanthpur
Traffic Volume             72039
Average Speed          89.790843
Incident Reports              10
dtype: object
```

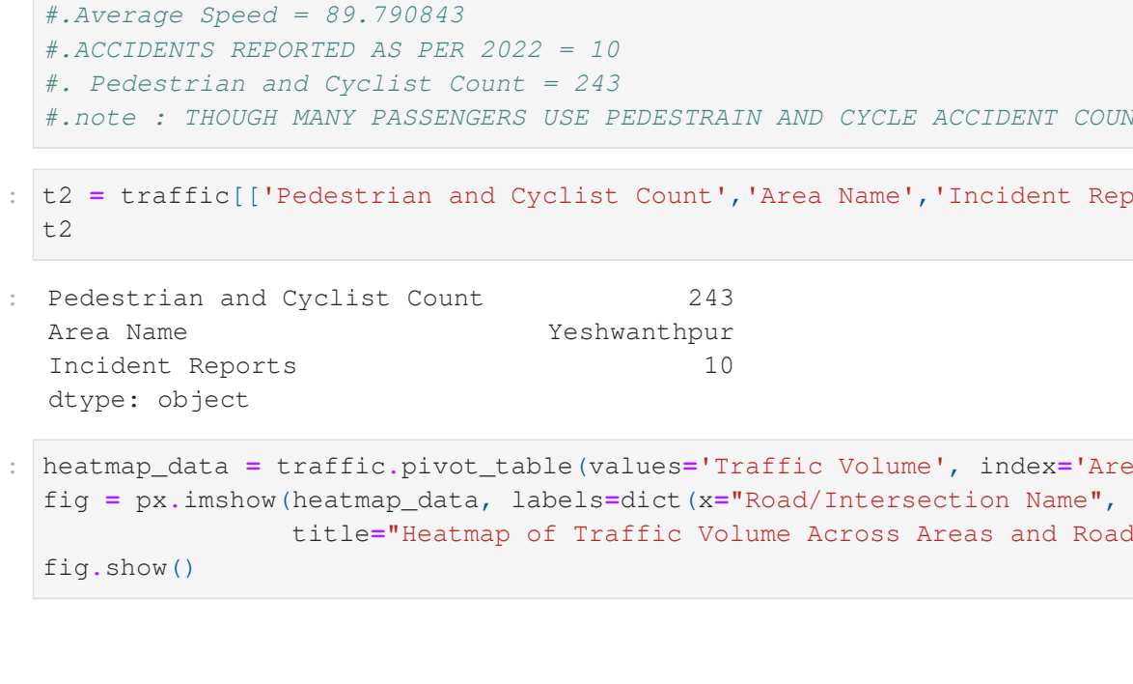
In [12]:
```
traffic[['Area Name','Traffic Volume','Average Speed','Incident Reports']].min()
```

Out[12]:
```
Area Name           Electronic City
Traffic Volume                 4233
Average Speed                  20.0
Incident Reports                  0
dtype: object
```

In [13]:
```
t1 = traffic[['Pedestrian and Cyclist Count','Area Name','Incident Reports']].head(10)
```

In [14]:
```
t1.plot.bar(y = 'Pedestrian and Cyclist Count')
```

Out[14]:
```
<Axes: >
```

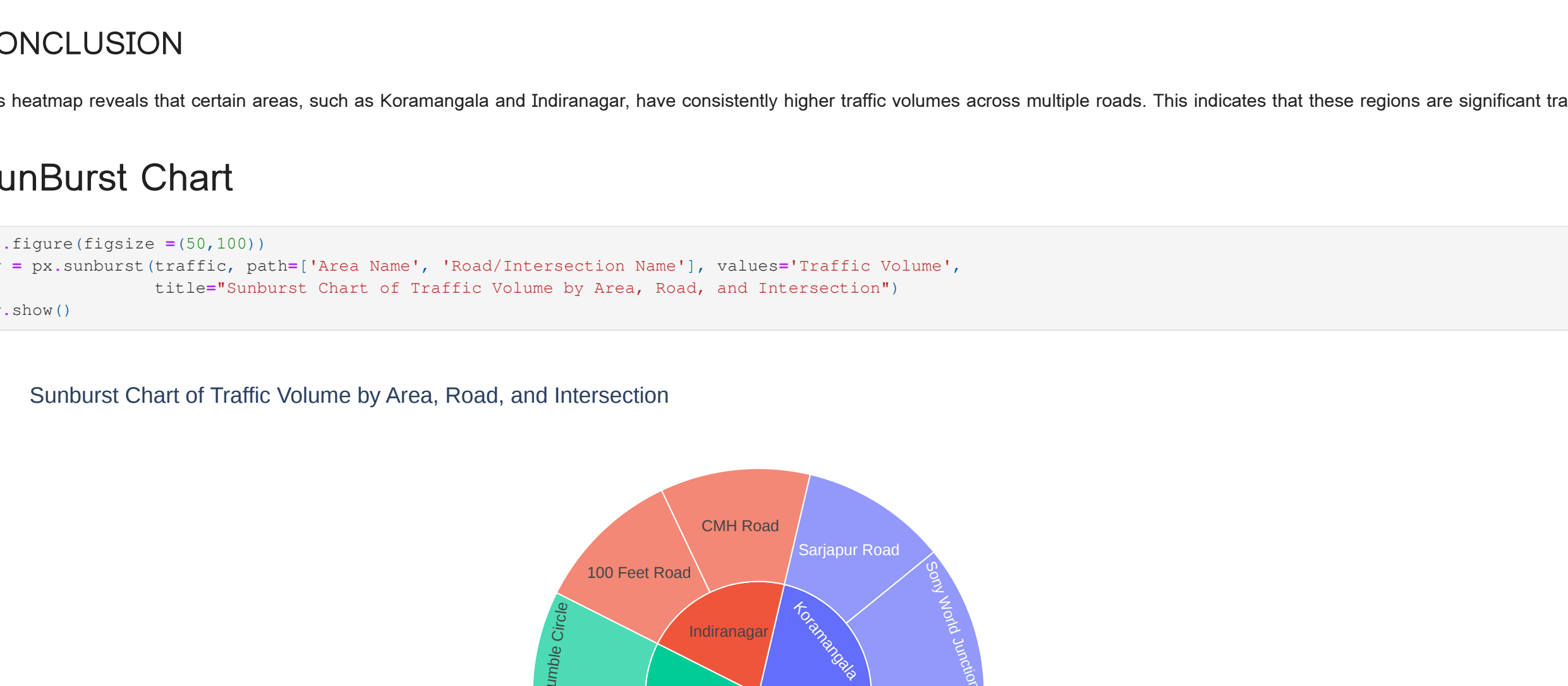


In [24]:
```
# CONCLUSION
#.TRAFFIC VOLUME IS MORE IN YESWANTHPUR
#.Traffic Volume = 72039
#.Average Speed = 89.790843
#.ACCIDENTS REPORTED AS PER 2022 = 10
#. Pedestrian and Cyclist Count = 243
#.note : THOUGH MANY PASSENGERS USE PEDESTRAIN AND CYCLE ACCIDENT COUNT IS MORE
```

In [16]:
```
t2 = traffic[['Pedestrian and Cyclist Count','Area Name','Incident Reports']].max()
t2
```

Out[16]:
```
Pedestrian and Cyclist Count             243
Area Name                       Yeshwanthpur
Incident Reports                          10
dtype: object
```

In [17]:
```
heatmap_data = traffic.pivot_table(values='Traffic Volume', index='Area Name', columns='Road/Intersection Name', aggfunc='sum')
fig = px.imshow(heatmap_data, labels=dict(x="Road/Intersection Name", y="Area Name", color="Traffic Volume"),
                title="Heatmap of Traffic Volume Across Areas and Roads")
fig.show()
```

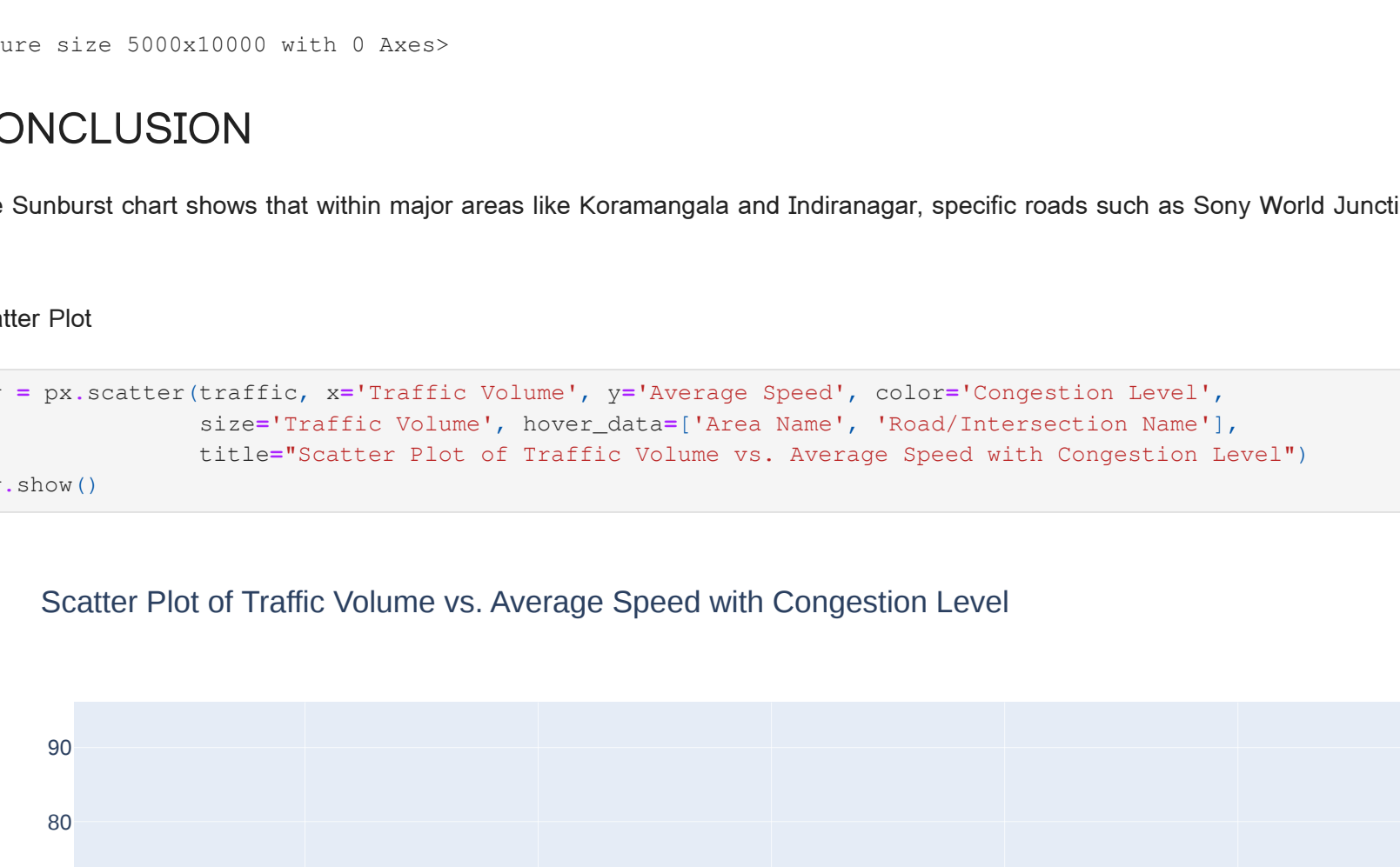


CONCLUSION

This heatmap reveals that certain areas, such as Koramangala and Indiranagar, have consistently higher traffic volumes across multiple roads. This indicates that these regions are significant traffic hubs.

SunBurst Chart

In [18]:
```
plt.figure(figsize =(50,100))
fig = px.sunburst(traffic, path=['Area Name', 'Road/Intersection Name'], values='Traffic Volume',
                  title="Sunburst Chart of Traffic Volume by Area, Road, and Intersection")
fig.show()
```



```
<Figure size 5000x10000 with 0 Axes>
```

CONCLUSION

The Sunburst chart shows that within major areas like Koramangala and Indiranagar, specific roads such as Sony World Junction and 100 Feet Road dominate traffic volumes, indicating key pressure points in these localities.

Scatter Plot

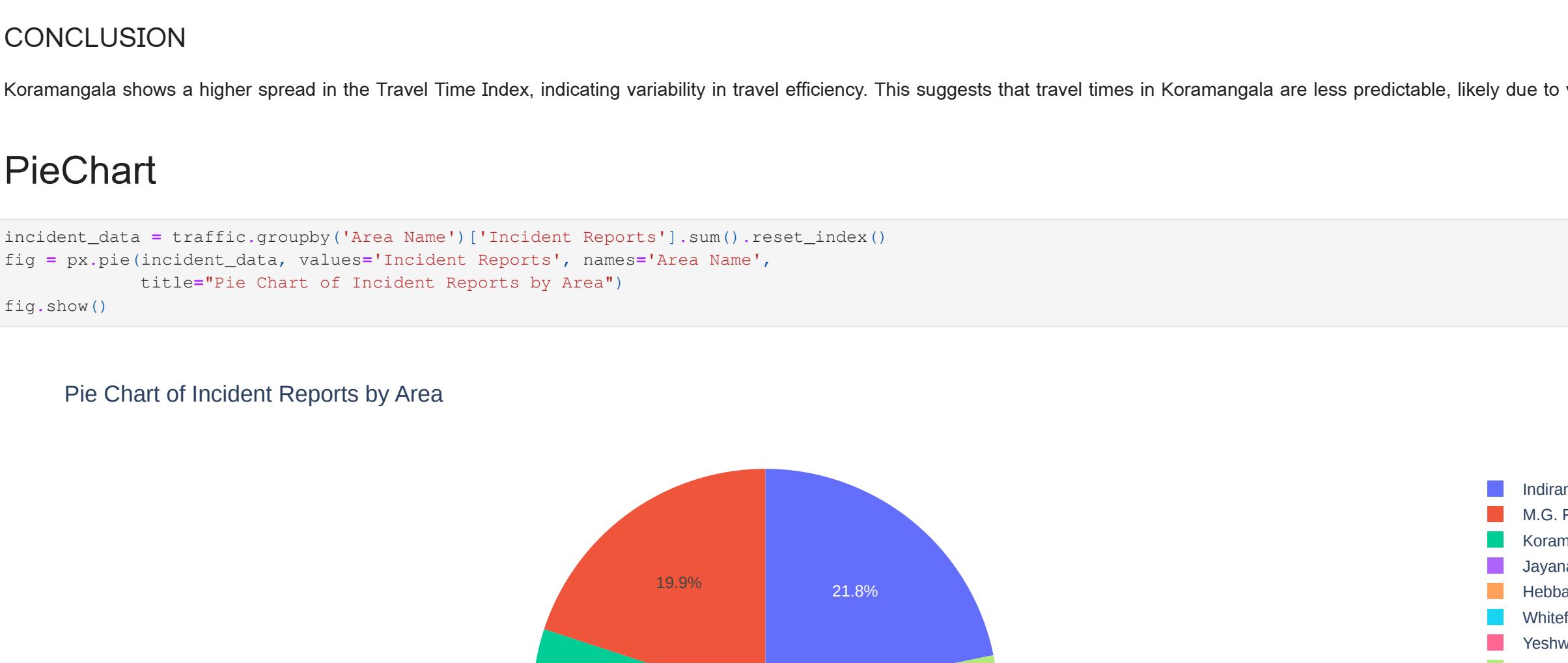
In [19]:
```
fig = px.scatter(traffic, x='Traffic Volume', y='Average Speed', color='Congestion Level',
                 size='Traffic Volume', hover_data=['Area Name', 'Road/Intersection Name'],
                 title="Scatter Plot of Traffic Volume vs. Average Speed with Congestion Level")
fig.show()
```

CONCLUSION

This plot shows an inverse relationship between traffic volume and average speed. Areas with high traffic volumes, like Indiranagar, tend to have lower speeds, which correlates with higher congestion levels.

BoxPlot

In [20]:
```
fig = px.box(traffic, x='Area Name', y='Travel Time Index',
             title="Box Plot of Travel Time Index by Area")
fig.show()
```

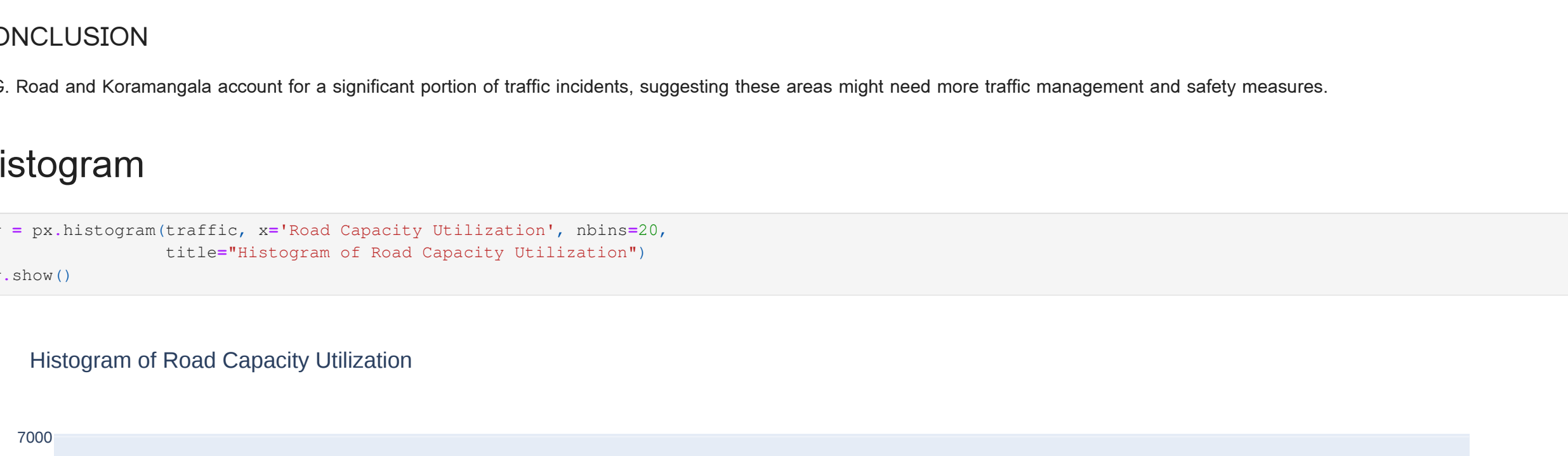


CONCLUSION

Koramangala shows a higher spread in the Travel Time Index, indicating variability in travel efficiency. This suggests that travel times in Koramangala are less predictable, likely due to varying traffic

PieChart

In [21]:
```
incident_data = traffic.groupby('Area Name')['Incident Reports'].sum().reset_index()
fig = px.pie(incident_data, values='Incident Reports', names='Area Name',
             title="Pie Chart of Incident Reports by Area")
fig.show()
```

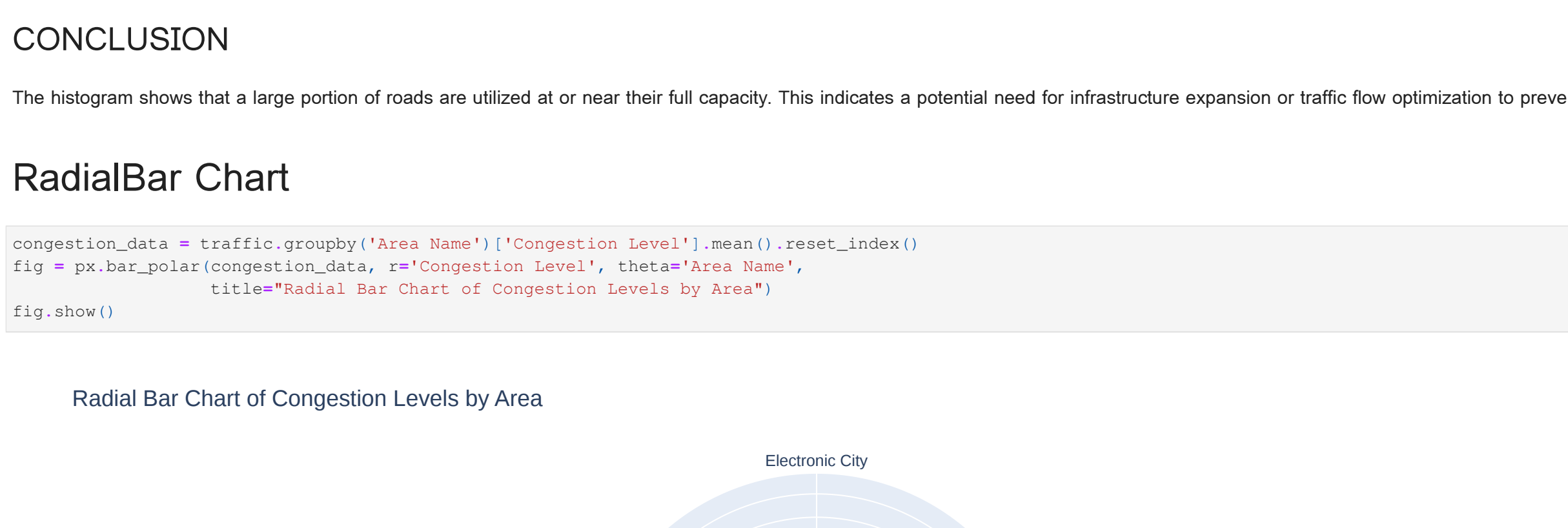


CONCLUSION

M.G. Road and Koramangala account for a significant portion of traffic incidents, suggesting these areas might need more traffic management and safety measures.

Histogram

In [22]:
```
fig = px.histogram(traffic, x='Road Capacity Utilization', nbins=20,
                   title="Histogram of Road Capacity Utilization")
fig.show()
```

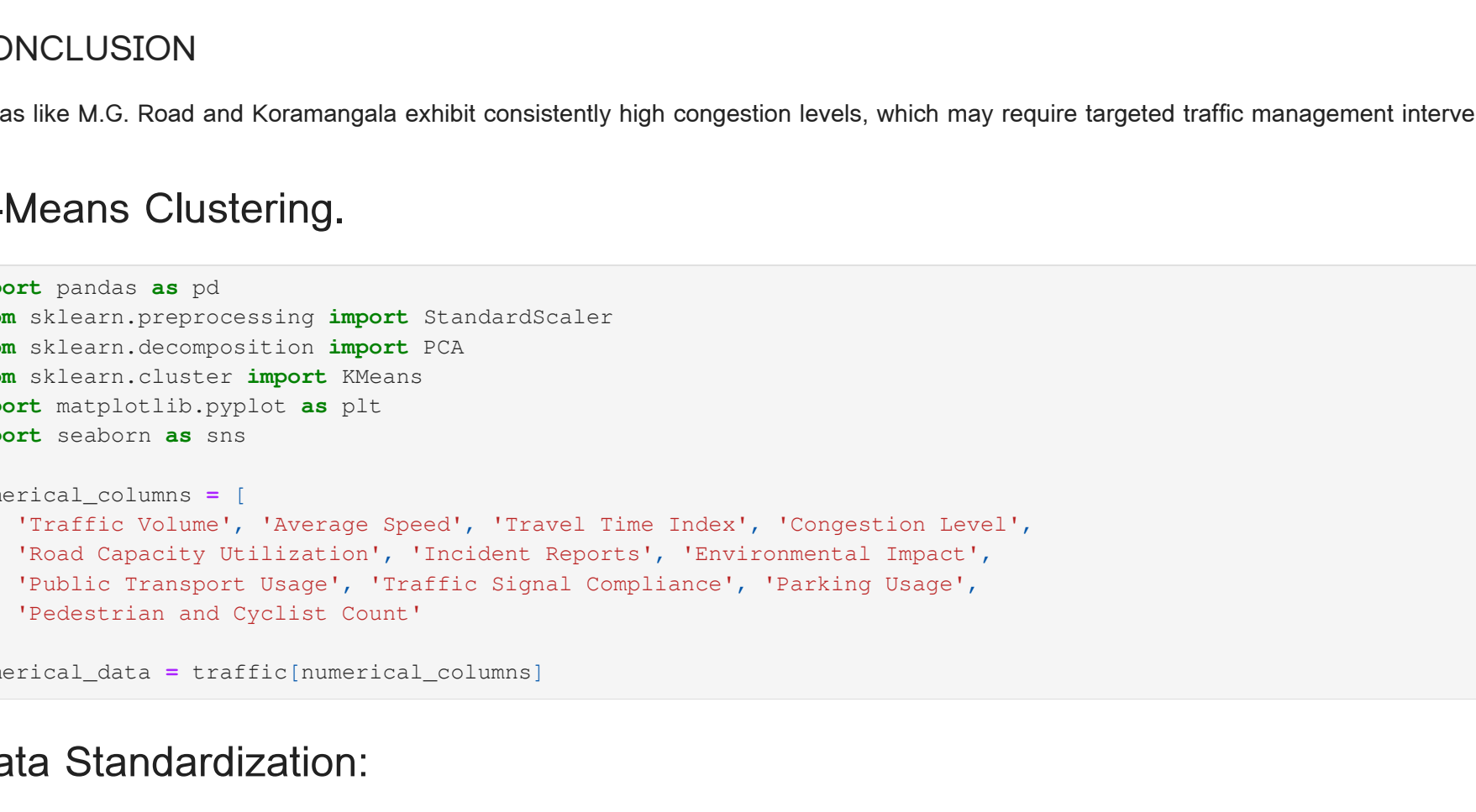


CONCLUSION

The histogram shows that a large portion of roads are utilized at or near their full capacity. This indicates a potential need for infrastructure expansion or traffic flow optimization to prevent overloading..

RadialBar Chart

In [23]:
```
congestion_data = traffic.groupby('Area Name')['Congestion Level'].mean().reset_index()
fig = px.bar_polar(congestion_data, r='Congestion Level', theta='Area Name',
                   title="Radial Bar Chart of Congestion Levels by Area")
fig.show()
```



CONCLUSION

Areas like M.G. Road and Koramangala exhibit consistently high congestion levels, which may require targeted traffic management interventions to alleviate pressure

K-Means Clustering.

In [28]:
```
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt
import seaborn as sns

numerical_columns = [
    'Traffic Volume', 'Average Speed', 'Travel Time Index', 'Congestion Level',
    'Road Capacity Utilization', 'Incident Reports', 'Environmental Impact',
    'Public Transport Usage', 'Traffic Signal Compliance', 'Parking Usage',
    'Pedestrian and Cyclist Count'
]
numerical_data = traffic[numerical_columns]
```

Data Standardization:

The numerical data is standardized using StandardScaler() to ensure all features have a mean of 0 and a standard deviation of 1.

Dimensionality Reduction with PCA:

Principal Component Analysis (PCA) is applied to reduce the dimensionality of the data to 2 components, which simplifies clustering.

In [ ]: